

Especificação do Projeto Prático: Sistema Gráfico Interativo (Estilo Asteroids)

Disciplina: Computação Gráfica

Docente: Tainá Isabela

Ambiente Tecnológico: Python (PyOpenGL / GLFW / Pygame)

1. Objetivo do Projeto

O objetivo deste trabalho prático é consolidar os conceitos teóricos de modelagem geométrica, transformações lineares, iluminação estática/dinâmica, mapeamento de texturas, interação em tempo real e detecção de colisões. Os alunos deverão desenvolver uma aplicação gráfica interativa inspirada no clássico jogo *Asteroids*, utilizando a linguagem **Python** e a API gráfica **OpenGL**.

2. Requisitos do Sistema (Entidades do Jogo)

O sistema deve gerenciar e renderizar de forma independente três categorias principais de entidades virtuais na cena:

A. Nave Espacial (Objeto Principal)

- **Controle e Orientação:** O objeto deve responder aos comandos do usuário, atualizando sua rotação e posição com base em vetores de aceleração/empuxo linear.
- **Propriedades Visuais:** Deve possuir uma malha poligonal própria e reagir adequadamente à iluminação do ambiente.

B. Projéteis (Disparos)

- **Dinâmica:** Devem ser inicializados a partir da posição atual da nave, herdando sua orientação angular e deslocando-se com vetor de velocidade constante.
- **Gerenciamento de Memória:** O sistema deve gerenciar múltiplos projéteis ativos, garantindo que sejam destruídos ao saírem dos limites do cenário visível para otimizar os recursos do pipeline.

C. Invasores / Obstáculos (Inimigos)

- **Comportamento:** Elementos que se movem de forma independente pela tela em trajetórias contínuas.
- **Mecânica:** O jogador deve desviar destes objetos ou interceptá-los utilizando os projéteis disparados pela nave.

3. Requisitos Técnicos Obrigatórios

A. Movimentação e Interação (Tempo Real)

- Implementação do laço principal do jogo (*Game Loop*) calculando as atualizações de posição das entidades com base no tempo decorrido.
- Suporte a entradas periféricas (teclado e mouse) para controle fluido da nave (movimentos de aceleração, desaceleração inercial e rotação).

B. Iluminação Dinâmica

- Configuração de um modelo clássico de iluminação (Ex: Phong ou Blinn-Phong) contendo as componentes ambiente, difusa e especular.
- Pelo menos uma fonte de luz (móvel ou direcional) que interaja corretamente com os vetores normais das superfícies de todas as malhas poligonais.

C. Mapeamento de Texturas

- Aplicação de coordenadas de textura (Coordenadas UV) sobre as malhas dos objetos ou no plano de fundo.
- Configuração de filtros de amostragem (como GL_LINEAR ou uso de Mipmaps) para evitar distorções visuais e serrilhados (aliasing).

D. Sistema de Detecção e Resposta a Colisões

O sistema deve avaliar de forma contínua as interseções no espaço utilizando volumes envoltórios simplificados:

1. Colisão Projétil × Invasor: Ao detectar o contato matemático entre o projétil e o inimigo, ambos devem disparar uma rotina de eliminação da tela e atualização da pontuação.

2. Colisão Invasor × Nave Espacial: O contato entre qualquer inimigo e a nave principal deve acionar imediatamente a rotina de perda de vida ou término da partida (*Game Over*).

Sugestão técnica: Utilizar testes baseados em **AABB** (*Axis-Aligned Bounding Boxes*) ou **Bounding Spheres/Circles** (checagem da distância euclidiana em relação aos raios dos objetos).

4. Diretrizes de Entrega

- **Código-Fonte:** Estruturado de forma limpa e modular em **Python**, utilizando corretamente bibliotecas como PyOpenGL para renderização e GLFW ou Pygame para gerenciamento de janela e eventos.
- **Código Limpo:** O código deve conter comentários explicativos sobre a organização do laço principal e da lógica das entidades, sem a necessidade de relatórios ou documentações extras focadas nas deduções matemáticas.